Litter — HTB SOC Sherlock Investigation

DNS Tunneling & Data Exfiltration Analysis

Scenario

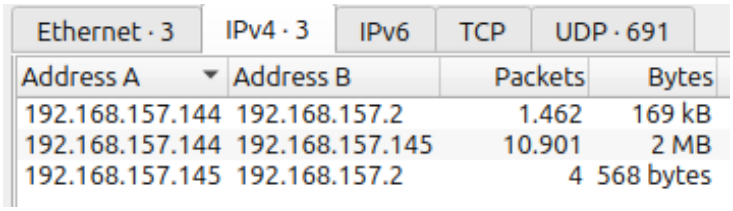
Khalid has just logged onto a host that he and his team use as a testing host for many different purposes. It's off their corporate network but has access to lots of resources on the network. The host is used as a dumping ground for a lot of people at the company, but it's very useful, so no one has raised any issues. Little does Khalid know; the machine has been compromised and company information that should not have been on there has now been stolen – it's up to you to figure out what has happened and what data has been taken.

Task 1: Suspect Protocol Identification

Question: At a glance, what protocol seems to be suspect in this attack?
Answer: `DNS`

Methodology

To identify the suspect protocol, we analyzed the conversation statistics and DNS traffic patterns. We examined the IPv4 conversation statistics:



Ethernet · 3 | IPv4 · 3 | IPv6 | TCP | UDP · 691

Address A	Address B	Packets	Bytes
192.168.157.144	192.168.157.2	1.462	169 kB
192.168.157.144	192.168.157.145	10.901	2 MB
192.168.157.145	192.168.157.2	4	568 bytes

Figure 1: IPv4 Conversation Statistics showing DNS-dominated traffic

Findings: The conversation between the victim (`192.168.157.144`) and the internal DNS resolver (`192.168.157.145`) accounted for **10,901 packets** — the vast majority of the capture. This is a significant anomaly, as legitimate DNS traffic would show many different hosts querying many different destinations.
In addition, the traffic is dominated by **TXT, MX, and CNAME** records — which are commonly abused for DNS tunneling and data exfiltration.

Task 2: Suspect Host IP

Question: There seems to be a lot of traffic between our host and another, what is the IP address of the suspect host?
Answer: `192.168.157.145`

Methodology

From the conversation statistics analysis, the host communicating most heavily with the victim (`192.168.157.144`) was identified as `192.168.157.145` with **10,901 packets** exchanged.

Task 3: First Command from Attacker

Question: What is the first command the attacker sends to the client?
Answer: `whoami`

Methodology

To find the first command sent by the attacker, we used a simple approach:

1. In Wireshark, filtered for DNS traffic from the attacker to the victim:

```
ip.src == 192.168.157.145 && ip.dst == 192.168.157.144 && dns
```

2. Right-clicked on any DNS response packet and selected **Follow → UDP Stream**.
3. Copied the raw hex data from the UDP stream.
4. Pasted the data into **CyberChef** and applied the **"From Hex"** operation.
5. Scrolled through the decoded output until finding the first readable command, which was `whoami` — a common reconnaissance command to identify the current user context.

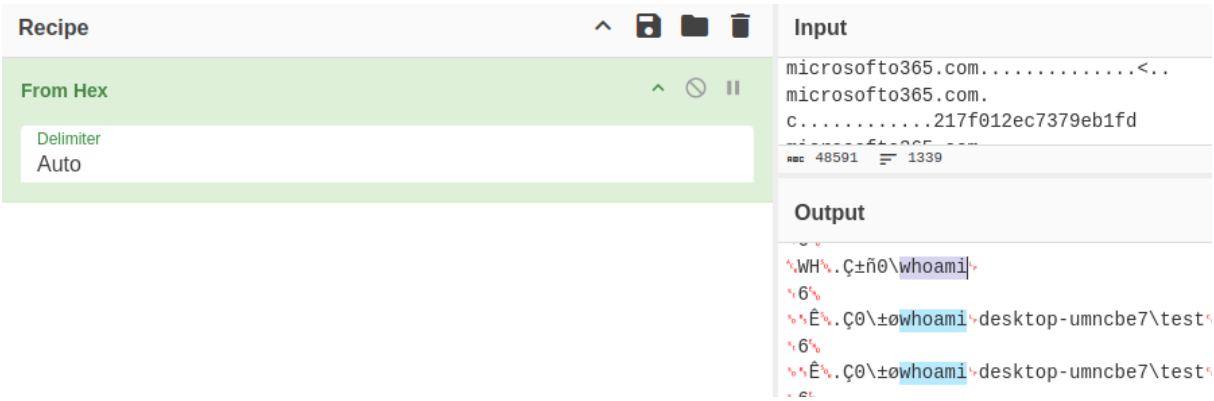


Figure 2: CyberChef Decoding — First Command `whoami`

Task 4: DNS Tunneling Tool Version

Question: What is the version of the DNS tunneling tool the attacker is using?
Answer: `0.07`

Methodology

We decoded all DNS TXT response payloads and searched for readable strings:

```
tshark -r suspicious_traffic.pcap -Y "dns.flags.response==1 && dns.txt
   " -T fields -e dns.txt | while read hex; do echo "$hex" | xxd -r -p
   ; done | strings
```

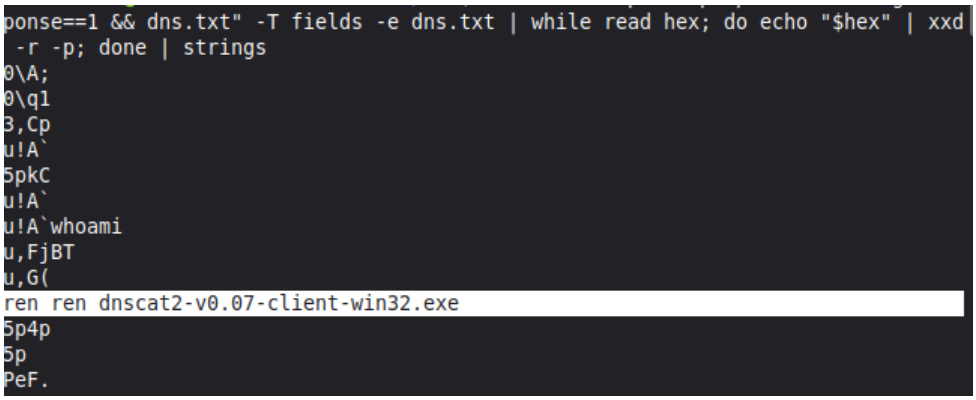


Figure 3: Decoded Strings showing dnscat2 version

Conclusion

The decoded output reveals:

- **Tool:** `dnscat2` — a well-known DNS tunneling tool
- **Version:** `v0.07`

Task 5: Renamed Tool

Question: The attackers attempts to rename the tool they accidentally left on the clients host. What do they name it to?
Answer: `win_installer.exe`

Methodology

To parse the tunneled traffic more effectively, we used a dnscat2 decoder script developed by josemlwdf[1]:

```
python3 dnscat_decoder.py suspicious_traffic.pcap "microsofto365.com"
    > dnscat_dump.txt
```

The `ren` (rename) command is a Windows built-in utility used to change the name of a file or directory. The output shows the attacker attempting to rename the tool multiple times:

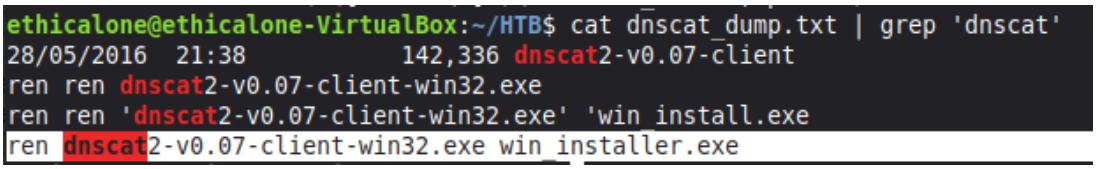


Figure 4: Rename command found in decoded traffic

The attacker renamed the tool to `win_installer.exe`.

[1] https://github.com/josemlwdf/DNScat-Decoder

Task 6: Cloud Storage Enumeration

Question: The attacker attempts to enumerate the users cloud storage. How many files do they locate in their cloud storage directory?
Answer: `0`

Methodology

We searched the decoded dnscat traffic for directory listings of the OneDrive folder.

OneDrive is Microsoft's built-in cloud storage solution, commonly used by users to store and sync personal files across devices — making it a natural target for an attacker looking to locate sensitive or stolen data.

```
cat dnscat_dump.txt | grep -A 10 "OneDrive"
```

```
C:\Users\test>
cd OneDrive

C:\Users\test\OneDrive>
ers\test\OneDrive

04/06/2021  08:52    <DIR>          .
04/06/2021  08:52    <DIR>
 ..
               0 File(s)              0 bytes
               2 Dir(s)  24,470,171,648 byt
es free

C:\Users\test\OneDrive>
  <DIR>          .vscode
11/06/2021  13:40    <DIR>          3D Objects
06/04/2021  12:42

              23 Dir(s)  24,548,376,576 bytes free
```

Figure 5: OneDrive Directory Listing

Conclusion

When the attacker listed the contents of `C:\Users\test\OneDrive`, the output showed:

- **0 File(s)** — no files present in the cloud storage directory.

Task 7: Stolen PII File Location

Question: What is the full location of the PII file that was stolen?
Answer: `C:\Users\test\Documents\client data optimisation\user details.csv`

Methodology

We manually searched the decoded dnscat traffic for file download attempts:

```
C:\Users\test\Documents\client data optimisation
window -i 3
'window' is not recognized as an internal or external command,
operable program or
 batch file.

C:\Users\test\Documents\client data optimisation>
download "C:\Users\test\Documents\client data optimisation\user details.csv"
'download' is not
whoami
desktop-umncbe7\test

C:\Users\test\Documents\client data optimisation>
```

Figure 6: Download Attempt — PII File

Conclusion

Although the `download` command failed (not recognized on Windows), it reveals the attacker's target file: **"C:\Users\test\Documents\client data optimisation\user details.csv"**

Task 8: Stolen PII Records Count

Question: Exactly how many customer PII records were stolen?
Answer: `721`

Methodology

We examined the content of the stolen file by decoding the exfiltrated data:

```
cat dnscat_dump.txt | grep -A 500 "user details.csv"
```

Output:

```
,job,company,ssn,residence,current_location,blood_group,website,username,name,sex,address,ma
0,Chief Technology Officer,Bennett Group,725-79-1073,"5021 Mary Glens...
1,Learning mentor,"Solis, Burns and Ward",384-57-7026,"097 Caleb Garden...
...
720,...
```

Interpretation

The CSV file contains:

- A header row with column names (`job, company, ssn, ...`)
- Data rows starting with an incremental ID (`0, 1, 2, 3, ...`)

The last visible record in the file is `720, ....` Since the IDs start at **0** and go up to **720**, the total number of records is:

$$720 + 1 = 721$$

Conclusion

The attacker stole **721 customer PII records**, each containing: Names, Social Security Numbers (SSN), Email addresses, Physical addresses, Job titles...

Summary of Findings

Task	Answer
Suspect Protocol	DNS
Suspect Host IP	192.168.157.145
First Command	whoami
DNS Tunneling Tool Version	0.07
Renamed Tool	win_installer.exe
Files in OneDrive	0
Stolen PII File Location	C:\Users\test\Documents\client data optimisation\user details.csv
Stolen PII Records Count	721

Conclusion

The investigation successfully identified:

- **DNS** as the suspect protocol used for malicious activity.
- The suspect host as `192.168.157.145`, acting as an internal DNS resolver forwarding tunneling queries.
- The attacker's first command as `whoami` — a reconnaissance command to identify the user context.
- The DNS tunneling tool as **dnscat2 version 0.07**.
- The attacker renamed the tool from `dnscat2-v0.07-client-win32.exe` to `win_installer.exe`.
- **0 files** were found in the victim's OneDrive cloud storage.
- The attacker targeted a PII file located at `C:\Users\test\Documents\client data optimisation\user details.csv`.
- A total of **721 customer PII records** were stolen.